

# RAG Pipeline offloading via NVMe-oF for Edge LLM Inference



202255570 양윤성 [youseong@pusan.ac.kr](mailto:youseong@pusan.ac.kr)

202455325 나예은 [nye6998@pusan.ac.kr](mailto:nye6998@pusan.ac.kr)

지도교수 안성용

---

## 목 차

|                                          |    |
|------------------------------------------|----|
| 1. 서론.....                               | 1  |
| 1.1. 연구 배경.....                          | 1  |
| 1.2. 연구 목표.....                          | 1  |
| 2. 연구 내용.....                            | 2  |
| 2.1. 요구사항 및 기존 구조 분석.....                | 3  |
| 2.1.1. 분리형 스토리지 환경의 처리 구조.....           | 3  |
| 2.1.2. 설계 요구사항.....                      | 3  |
| 2.2. 전체 시스템 구조 설계.....                   | 4  |
| 2.3. 서버 측 RAG 파이프라인 설계.....              | 5  |
| 2.3.1. 문서 및 벡터 인덱스 구성.....               | 5  |
| 2.3.2. 질의 검색과 프롬프트 구성.....               | 5  |
| 2.3.3. 토큰화와 LLM 입력 형식.....               | 6  |
| 2.4. NVME-oF 기반 질의 전달 결과 및 결과 반환 설계..... | 6  |
| 2.4.1. 오프로딩 요청과 결과 저장 영역.....            | 6  |
| 2.4.2. 질의 처리 순서.....                     | 7  |
| 2.4.3. 구현 시 고려할 제약사항.....                | 7  |
| 2.5. Edge 측 LLM 연동 및 구현 환경 계획.....       | 8  |
| 2.6. 현재 진행 단계 및 향후 구현 계획.....            | 9  |
| 3. 결론 및 기대 효과.....                       | 10 |

---

## 1. 서론

### 1.1. 연구 배경

최근 대규모 언어 모델(LLM, Large Language Model)은 다양한 분야에서 활용되고 있으나, 사실과 다른 내용을 생성하는 환각 현상과 최신 정보를 반영하기 어렵다는 한계가 있다. 검색 증강 생성(RAG, Retrieval-Augmented Generation)은 외부 문서나 데이터베이스에서 검색한 관련 정보로 모델의 입력을 보강하여 이러한 한계를 완화하는 기술이다. 이를 통해 모델을 재학습하지 않고도 최신 정보나 특정 분야의 지식을 응답 생성에 활용할 수 있다.

그러나 RAG에 사용하는 Vector DB의 규모가 커질수록 저장 공간과 데이터 접근에 대한 부담도 증가한다. 특히 자원이 제한된 Edge Device에서는 대규모 벡터 데이터와 인덱스를 로컬에 저장하고 관리하기 어려울 수 있다. 이러한 제약에 대응하는 방법으로, 컴퓨팅과 저장 공간을 분리하고 벡터 데이터와 인덱스를 원격 Storage Server에 배치하는 분리형 스토리지 아키텍처를 활용할 수 있다.

이러한 구성에서 RAG의 검색 연산을 Edge Device에서 수행하면, 검색 과정에서 원격 스토리지의 인덱스와 데이터에 반복적으로 접근하게 된다. 이때 네트워크를 통한 데이터 이동과 요청·응답에 따른 통신 지연이 누적되어 RAG 처리 성능에 영향을 줄 수 있다. 따라서 Edge 환경에서 RAG를 활용할 때에는 장치의 저장 용량뿐 아니라, 원격 데이터 접근에 따른 전송 부담과 처리 지연도 함께 고려해야 한다.

### 1.2. 연구 목표

본 연구의 목적은 자원이 제한된 Edge Device에서 대규모 RAG 데이터베이스를 효율적으로 활용하기 위해, NVMe over Fabrics(NVMe-oF) 기반의 RAG 파이프라인 오프로딩 구조를 설계하고 구현하는 것이다. 대규모 Vector DB를 원격 Storage Server에 저장하고 Edge에서 RAG를 수행하는 환경에서는, 검색 과정에서 원격 인덱스에 반복적으로 접근하여 네트워크를 통한 데이터 이동과 처리 지연이 발생한다. 이를 개선하기 위해 데이터가 저장된 Storage Server에서 RAG 파이프라인과 tokenization을 수행하고, Edge에서는 반환된 토큰 시퀀스를 이용해 LLM 추론을 수행하는 Near-Data Processing 구조를 적용하고자 한다. 이를 통해 원격 인덱스 접근에 따른 데이터 이동을 줄이고, Edge 환경에서 RAG 기반 LLM 서비스의 처리 효율을 높이는 것을 목표로 한다.

---

구체적인 연구 목적은 다음과 같다.

1. **Edge와 Storage Server 간 RAG 파이프라인 오프로딩 구조 설계**

질의 임베딩, 유사도 검색, 검색 결과를 반영한 프롬프트 구성 및 tokenization을 Storage Server로 오프로딩하고, Edge에서는 LLM 추론을 수행하도록 역할을 분리한다.

2. **NVMe-oF 기반 오프로딩 경로 구현**

NVMe-oF와 SPDK를 활용하여 Edge의 질의를 Storage Server에 전달하고, 서버에서 처리한 토큰 시퀀스를 Edge가 읽을 수 있도록 반환하는 사용자 정의 NVMe 명령 기반의 처리 경로를 구현한다.

3. **Storage Server의 RAG 처리 결과와 Edge의 LLM 추론 연동**

Storage Server에서 생성한 토큰 시퀀스를 Edge의 LLM 입력으로 직접 연결하여, 질의 전달부터 검색, tokenization, 응답 생성까지 이어지는 통합 시스템을 구현한다.

4. **데이터 이동량 및 처리 지연 감소 효과 검증**

Edge에서 원격 인덱스에 접근하며 RAG를 수행하는 기존 방식과 제안 구조를 비교하여, Edge-Storage Server 간 데이터 이동량과 RAG 단계별 처리 지연을 측정하고 오프로딩의 효과를 평가한다.

## 2. 연구 내용

본 연구는 NVMe-oF 기반 분리형 스토리지 환경에서 RAG 파이프라인의 실행 위치를 Storage Server로 옮기는 구조를 대상으로 한다. 현재까지 Edge Device와 Storage Server의 역할, 질의 전달과 결과 반환 절차, LLM 입력 연동 방안을 이론적으로 구성하였다. 본 단락에서는 이러한 설계 내용과 구현 시 고려할 제약사항을 설명한다. 실제 시스템 구현과 측정은 향후 수행할 예정이다.

---

## 2.1. 요구사항 및 기존 구조 분석

### 2.1.1. 분리형 스토리지 환경의 처리 구조

Baseline은 문서와 벡터 인덱스를 Storage Server에 저장하고, RAG 처리와 LLM 추론을 Edge Device에서 수행하는 구조이다. Edge는 NVMe-oF를 통해 원격 인덱스에 접근하여 관련 문서를 검색한 뒤, 프롬프트 구성과 토큰화를 거쳐 답변을 생성한다.

이 구조에서는 검색 인덱스가 Edge의 메모리에 충분히 적재되지 못할 경우, 검색 과정의 데이터 접근이 NVMe-oF를 통한 원격 I/O로 이어질 수 있다. 따라서 전체 전송량뿐 아니라 반복적인 I/O 요청과 응답에 소요되는 시간도 고려해야 한다. 본 연구는 검색에 필요한 데이터 접근을 서버 내부에서 처리하도록 실행 위치를 변경하여 이러한 부담을 줄이는 방향으로 설계하였다.

### 2.1.2. 설계 요구사항

제안 구조에서는 대규모 Vector DB를 서버에 배치하고, Edge에는 질의 요청과 LLM 추론에 필요한 기능을 유지한다. 이를 위해 다음과 같은 요구사항을 설정하였다.

| 요구사항          | 설계 방향                                       |
|---------------|---------------------------------------------|
| RAG 처리의 서버 배치 | 질의 임베딩, 유사도 검색, 프롬프트 구성 및 토큰화를 서버에서 수행한다.   |
| NVMe-oF 경로 활용 | 사용자 정의 NVMe 명령으로 질의와 결과 저장 위치를 전달한다.        |
| LLM 입력 연동     | 서버가 반환한 입력 토큰 ID를 Edge의 LLM 추론 과정에 연결한다.    |
| 결과의 정확한 전달    | 결과 길이와 처리 상태를 확인하고, 저장 위치와 캐시 일관성을 관리한다.    |
| 비교 조건의 일치     | 기존 방식과 제안 방식에 동일한 문서, 질의, 모델 및 검색 설정을 적용한다. |

## 2.2. 전체 시스템 구조 설계

전체 시스템은 Edge Device와 Storage Server로 구성한다. Edge는 사용자 질의를 받아 오프로딩을 요청하고, 서버에서 반환한 토큰 시퀀스를 이용하여 답변을 생성한다. Storage Server는 Vector DB와 RAG 처리 모듈을 함께 배치하여, 유사도 검색 중 필요한 인덱스와 문서 접근을 서버 내부에서 처리하도록 구성한다. 두 장치 사이의 통신에는 NVMe-oF/TCP를 적용할 계획이다.

오프로딩 범위는 질의 임베딩부터 토큰화까지로 설정하였다. 유사도 검색만 서버로 옮기는 경우 검색 문서를 Edge로 전달한 뒤 프롬프트 구성과 토큰화를 수행해야 한다. 본 설계에서는 이 후속 처리도 서버에 배치하고, LLM 입력에 사용할 토큰 ID 시퀀스를 반환하도록 한다. 이를 통해 Edge가 검색 인덱스를 반복적으로 읽는 경로와 검색 문서를 다시 전처리하는 과정을 줄이고자 한다.

| 구분        | Edge Device                     | Storage Server                         |
|-----------|---------------------------------|----------------------------------------|
| 주요 역할     | 질의 입력, 오프로딩 요청, LLM 추론          | Vector DB 관리 및 RAG 처리                  |
| 질의 처리     | 질의와 결과 저장 위치 전달                 | 질의 임베딩 및 유사도 검색                        |
| LLM 입력 준비 | 반환된 토큰 ID와 길이 확인                | 프롬프트 구성 및 토큰화                          |
| 결과 처리     | 결과 파일을 읽어 추론 입력으로 전달            | 지정된 저장 영역에 토큰 기록                       |
| 주요 구성 요소  | NVMe-oF 클라이언트, 파일시스템, llama.cpp | SPDK NVMe-oF target, Vector DB, RAG 모듈 |

본 연구의 Near-Data Processing은 저장장치에 전용 연산 하드웨어를 추가하는 방식이 아니라, 데이터가 위치한 서버의 연산 자원을 활용하는 방식으로 구성한다. 성능 개선의 주된 가설은 원격 인덱스 접근을 서버 내부의 접근으로 바꾸는 데 있다. 토큰화의 서버 배치가 전송량과 처리 시간에 미치는 추가 효과는 구현 이후 별도로 확인할 필요가 있다.

---

## 2.3. 서버 측 RAG 파이프라인 설계

### 2.3.1. 문서 및 벡터 인덱스 구성

검색에 사용할 문서는 사전에 일정 단위로 분할하고, 각 문서 조각에 대한 임베딩 벡터를 생성하여 원문과 함께 저장할 계획이다. Vector DB는 PostgreSQL과 pgvectorscale을 활용하는 구성을 검토하였다. 질의마다 전체 문서를 비교하는 부담을 줄이기 위해 디스크 기반 벡터 검색 인덱스를 활용하고, 데이터베이스와 검색 모듈을 Storage Server에 함께 배치한다.

문서 임베딩과 질의 임베딩에는 동일한 모델 및 호환되는 전처리 설정을 적용한다. 초기 임베딩 모델은 bge-m3로 계획하였다. 문서 분할 크기, 검색 결과 개수 및 인덱스 설정은 검색 품질과 처리량에 영향을 주므로, 향후 실험에서 두 방식에 공통으로 적용할 조건으로 관리할 예정이다. 문서 임베딩과 인덱스 구축은 질의 처리에 앞선 준비 단계로 구분한다.

### 2.3.2. 질의 검색과 프롬프트 구성

Storage Server가 오프로딩 요청을 받으면 질의 문자열을 임베딩 벡터로 변환하고, Vector DB에서 유사도가 높은 문서 조각을 검색하도록 설계하였다. 검색 과정에서 필요한 인덱스 탐색과 문서 읽기는 서버의 저장장치에 접근하여 처리한다. Edge는 검색 중간 결과나 인덱스 블록을 직접 요청하지 않고, 서버의 처리 완료를 기다리는 구조로 구성한다.

검색이 끝나면 원래 질의와 검색된 문서 내용을 결합하여 LLM 입력 프롬프트를 구성한다. 이때 모델이 허용하는 입력 길이를 넘지 않도록 검색 문서의 개수와 포함할 분량을 제한하는 방안이 필요하다. 기존 방식과 제안 방식에는 동일한 프롬프트 형식과 문서 선택 기준을 적용하여, 처리 위치의 차이를 비교할 수 있도록 할 계획이다.

---

### 2.3.3. 토큰화와 LLM 입력 형식

서버는 구성된 프롬프트를 Edge의 LLM과 호환되는 토큰라이저로 변환하고, 결과를 입력 토큰 ID의 순서열로 저장하도록 설계하였다. 이때 반환하는 것은 LLM이 생성한 답변 토큰이 아니라, 질의와 검색 문서를 반영한 입력 토큰이다. 답변 생성에 필요한 모델 추론은 Edge에서 수행한다.

서버와 Edge 사이에는 토큰라이저 버전, 어휘 집합, 특수 토큰 처리 및 프롬프트 형식이 일치해야 한다. 따라서 서버에서 생성한 토큰 ID와 동일한 프롬프트를 Edge에서 토큰화한 결과가 일치하는지 확인하는 절차를 구현 단계에 포함할 계획이다. 토큰 ID의 자료형과 바이트 순서, 유효한 토큰 개수도 결과 형식에 함께 정의하여 추론 입력의 해석이 달라지지 않도록 한다.

## 2.4. NVMe-oF 기반 질의 전달 결과 및 결과 반환 설계

### 2.4.1. 오프로딩 요청과 결과 저장 영역

일반적인 블록 읽기 및 쓰기 요청과 RAG 실행 요청을 구분하기 위해 사용자 정의 NVMe 명령을 사용하는 방안을 설계하였다. 명령에는 사용자 질의와 결과를 기록할 저장 영역 정보를 전달하고, Storage Server의 SPDK NVMe-oF target에서 해당 요청을 RAG 처리 모듈로 연결하도록 할 계획이다. 실제 명령 코드와 세부 필드 형식은 구현 단계에서 확정할 예정이다.

결과 반환에는 placeholder 파일을 사용하는 구조를 검토하였다. 이 파일은 Edge가 사용하는 원격 NVMe 블록 장치의 파일시스템에 미리 생성하는 결과 수신용 파일이다. Edge는 파일의 저장 영역을 확보한 뒤 FIEMAP을 통해 파일의 물리적 배치 정보를 확인하고, 이를 서버가 접근하는 NVMe namespace의 논리 블록 주소(LBA) 범위로 대응시키는 방식을 사용할 계획이다. 서버는 파일 경로가 아니라 전달받은 저장 영역에 결과를 기록한다.

파일이 여러 영역으로 나뉘어 저장될 수 있으므로, 단일 시작 주소만 가정하지 않고 각 영역의 주소와 길이를 처리해야 한다. 또한 파일시스템의 물리적 오프셋과 NVMe namespace의 LBA가 같은 기준을 사용하도록 장치 구성과 파티션 오프셋을 확인해야 한다. 결과 크기를 고려하여 충분한 공간을 확보하고, 처리 도중 파일 크기나 저장 위치가 바뀌지 않도록 관리하는 것을 구현 조건으로 설정하였다.



---

### 2.4.2. 질의 처리 순서

하나의 질의에 대한 처리 흐름은 다음과 같이 구성하였다. 각 단계는 향후 구현할 절차이며, 현재 실제 명령 전송이나 결과 반환 동작을 검증한 상태는 아니다.

#### 1단계 결과 영역 준비

Edge에서 결과 수신용 placeholder 파일을 생성하고 필요한 저장 공간을 확보한다. 파일의 실제 할당 상태를 확인한 뒤, 결과를 기록할 LBA 범위와 수용 가능한 크기를 구한다.

#### 2단계 오프로딩 요청

Edge에서 질의와 결과 저장 영역 정보를 사용자 정의 NVMe 명령으로 전달한다. 서버는 요청 형식과 저장 영역의 범위를 확인하고 RAG 처리를 시작하도록 한다.

#### 3단계 서버의 RAG 처리

서버에서 질의 임베딩, 유사도 검색, 프롬프트 구성 및 토큰화를 순서대로 수행한다. 인덱스와 문서에 대한 접근은 서버 내부에서 처리한다.

#### 4단계 결과 기록과 완료 통지

서버에서 토큰 시퀀스를 지정된 LBA 범위에 기록한다. 결과 쓰기가 완료된 뒤 처리 상태와 유효한 결과 길이를 Edge에 알리도록 한다.

#### 5단계 결과 읽기와 추론

Edge에서 완료 상태를 확인하고 placeholder 파일을 Direct I/O로 읽는다. 반환된 토큰 개수와 형식을 검사한 뒤 llama.cpp의 추론 입력으로 전달하여 답변을 생성하도록 한다.

### 2.4.3. 구현 시 고려할 제약사항

서버가 파일시스템을 거치지 않고 결과 영역에 기록하면, Edge의 파일 캐시에 남은 내용과 실제 저장장치의 내용이 달라질 수 있다. 이를 줄이기 위해 placeholder 파일 접근에 Direct I/O를 적용하고, 공간을 확보하는 과정에서는 실제 쓰기를 통해 파일 영역을 초기

화하는 방안을 검토하였다. 다만 Direct I/O만으로 모든 일관성 문제가 해결된다고 가정하지 않고, 초기화 완료 시점과 서버 쓰기 완료 시점, 결과 읽기 순서를 함께 검증할 예정이다.

반환되는 토큰 시퀀스의 길이는 질의와 검색 문서에 따라 달라지므로, 파일 용량을 초과하는 결과에 대한 처리도 필요하다. 서버는 결과가 지정된 범위에 들어가는지 확인하고, Edge는 결과 길이와 완료 상태를 검증한 뒤 유효한 데이터만 읽도록 설계한다. 블록 크기에 따른 I/O 정렬과 패딩, 통신 실패 및 잘못된 주소 전달에 대한 오류 처리는 구현 단계의 확인 항목으로 남겨 두었다.

## 2.5. Edge 측 LLM 연동 및 구현 환경 계획

Edge의 LLM 실행 환경은 Jetson Orin Nano에서 llama.cpp를 사용하는 구성을 계획하였다. 서버가 토큰화를 수행하므로 Edge에서는 동일한 프롬프트를 다시 토큰화하지 않고, 반환된 토큰 ID를 모델 입력으로 전달하는 경로를 구성할 예정이다. 이후 프리필과 답변 토큰 생성 등 LLM 추론은 Edge에서 진행하도록 한다.

초기 구현 및 평가 환경은 다음과 같이 계획하였다. 아래 항목은 구축을 완료한 환경이 아니라 구현을 위한 구성안이며, 데이터 규모와 세부 설정은 실제 장치의 자원 및 호환성을 확인하면서 확정할 예정이다.

| 항목             | 계획한 구성                             |
|----------------|------------------------------------|
| Edge Device    | Jetson Orin Nano                   |
| Storage Server | SSD 및 SPDK NVMe-oF target을 사용하는 서버 |
| 전송 경로          | NVMe-oF/TCP, 초기 1 Gbps 유선 네트워크     |
| Vector DB      | PostgreSQL 및 pgvectorscale         |
| 임베딩 모델         | bge-m3                             |
| LLM 및 실행 도구    | Gemma 3 270M 및 llama.cpp           |
| 평가용 워크로드       | TriviaQA 질의 및 검색용 문서               |
| 데이터 규모         | 문서와 벡터 인덱스를 포함하여 약 7 GB 규모를 우선 검토  |

---

## 2.6. 현재 진행 단계 및 향후 구현 계획

현재까지 진행한 내용은 기존 구조의 원격 데이터 접근 문제를 정리하고, 서버 측 RAG 처리와 Edge 측 LLM 추론을 연결하는 시스템 구성을 설계한 것이다. 질의 전달, 결과 저장 영역 확보, 서버의 토큰 기록 및 Edge의 결과 읽기 순서를 구체화하였다. SPDK 사용자 정의 명령 처리, 서버 RAG 모듈, llama.cpp 연동은 아직 구현하지 않았으며, 데이터 이동량이나 지연시간에 대한 측정 결과도 없는 상태이다.

향후에는 먼저 서버의 RAG 파이프라인을 독립적으로 구성하여 질의에 대한 검색 결과와 입력 토큰을 확인할 계획이다. 이어서 NVMe-oF 통신 환경과 사용자 정의 명령 처리 경로를 구현하고, placeholder 파일을 통한 결과 반환이 정확하게 이루어지는지 검증한다. 이 경로가 확인되면 Edge의 llama.cpp에 토큰 입력을 연결하여 질의 입력부터 답변 생성까지 통합할 예정이다.

통합 후에는 Edge에서 원격 인덱스에 접근하는 기존 방식과 제안 방식을 같은 조건에서 비교할 계획이다. 주요 평가 항목은 Edge와 Storage Server 사이의 데이터 이동량, 질의 임베딩·유사도 검색·토큰화의 단계별 처리 시간, 요청부터 입력 토큰 확보까지의 지연 시간으로 설정하였다. LLM 생성 시간을 포함한 전체 응답시간은 RAG 처리 시간과 구분하여 측정하고, NVMe-oF 명령 전송과 결과 파일 준비 및 읽기에 드는 비용도 평가에 포함할 예정이다.

두 방식에 동일한 질의 집합과 모델을 사용하고, 검색 결과 개수 및 캐시 조건을 통제하여 비교할 계획이다. 검색 결과와 입력 토큰의 일치 여부를 먼저 확인한 뒤 성능을 측정하고, 필요한 경우 검색만 오프로딩하는 구성과 토큰화까지 포함한 구성을 나누어 비교한다. 이를 통해 데이터 접근 위치 변경의 효과와 토큰화 오프로딩의 추가 효과를 구분하여 분석할 예정이다.

---

### 3. 결론 및 기대 효과

본 연구에서는 자원이 제한된 Edge Device에서 RAG 기반 LLM 서비스를 효율적으로 수행하기 위해, NVMe-oF와 SPDK를 활용한 RAG 파이프라인 오프로딩 구조와 주요 처리 흐름을 설계하였다. 향후 질의 임베딩, 유사도 검색, 프롬프트 구성 및 토큰화를 데이터가 저장된 Storage Server에서 수행하고, Edge Device에서는 반환된 토큰 시퀀스를 이용하여 LLM 추론을 수행하도록 구현할 예정이다. 이를 위해 사용자 정의 NVMe 명령을 기반으로 질의 전달과 처리 결과 반환 경로를 구성하고, 서버의 RAG 처리 결과를 Edge Device의 LLM 입력에 연동할 계획이다.

이러한 구조를 통해 다음과 같은 효과가 기대된다.

#### 1. 데이터 이동량 및 원격 I/O 오버헤드 감소

유사도 검색 과정에서 필요한 인덱스 접근을 Storage Server 내부에서 처리함으로써, Edge Device가 원격 데이터에 반복적으로 접근하는 부담을 줄일 수 있을 것으로 예상된다. 이를 통해 두 장치 간 데이터 이동량과 원격 I/O 오버헤드를 줄이고, 검색 단계의 처리 지연을 완화할 것으로 기대된다.

#### 2. Edge Device의 연산 부담 완화

RAG 검색과 전처리를 Storage Server에서 수행하면 Edge Device의 연산 부담이 줄어들 것으로 예상된다. 이에 따라 제한된 자원을 LLM 추론에 활용할 수 있어, 자원이 부족한 Edge 환경에서도 RAG 기반 서비스를 보다 효율적으로 운영할 수 있을 것으로 기대된다.

#### 3. 범용 하드웨어 기반의 적용 가능성 확대

NVMe-oF와 SPDK를 활용하여 별도의 전용 가속 하드웨어 없이 구현할 수 있는 구조를 지향한다. 이를 통해 대규모 Vector DB를 활용해야 하는 다양한 Edge 환경에 적용할 수 있을 것으로 기대된다.

현재는 기본 구조와 주요 처리 흐름을 설계한 단계이며, 향후 구현을 통해 기존 방식과 제안 구조의 데이터 이동량 및 RAG 단계별 처리 시간을 비교하여 이러한 기대 효과를 검증할 예정이다.